

Segment tree

```
#include <bits/stdc++.h>
using namespace std;

/* Segment Tree
 *Description:* Range queries, build  $O(n)$ , query and update  $O(\log n)$ ,
positions  $[0, n - 1]$ 
 *Status:* Highly tested
 */
template<class T, T m_(T, T)> struct segment_tree{
    int n; vector<T> ST;
    segment_tree(){}
    segment_tree(vector<T> &a){
        n = a.size(); ST.resize(n << 1);
        for (int i=n;i<(n<<1);i++)ST[i]=a[i-n];
        for (int i=n-1;i>0;i--)ST[i]=m_(ST[i<<1],ST[i<<1|1]);
    }
    void update(int pos, T val){ // replace with val
        ST[pos += n] = val;
        for (pos >>= 1; pos > 0; pos >>= 1)
            ST[pos] = m_(ST[pos<<1], ST[pos<<1|1]);
    }
    T query(int l, int r){ // [l, r]
        T ansL, ansR; bool hasL = 0, hasR = 0;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1)
                ansL=(hasL?m_(ansL,ST[l++]):ST[l++]),hasL=1;
            if (r & 1)
                ansR=(hasR?m_(ST[--r],ansR):ST[--r]),hasR=1;
        }
        if (!hasL) return ansR; if (!hasR) return ansL;
        return m_(ansL, ansR);
    }
};

int merge(int a, int b){
    return max(a,b);
}

int main (){
    int n = 10;
    vector<int> values(n);
    values.assign(n,2);
    segment_tree<int, merge> sTree(values);

    return 0;
}
```

Grafo

```
#include <bits/stdc++.h>
using namespace std;

typedef vector<vector<pair<int, int>>> graph;

int main()
{
    int n = 0, c = 0;
    cin >> n >> c;
    graph g = {
        {},
        {{2, 2}, {3, 1}, {4, 2}},
        {{4, 1}}
    };
    g.erase();
    for (int i = 0; i < c; i++)
    {
        int n1 = 0, n2 = 0, n3 = 0;
        cin >> n1 >> n2 >> n3 ;
        g[n1].push_back({n2, n3});
    }
    for (int j = 0; j < g.size(); j++)
    {
        for (int i = 0; i < g[j].size(); i++)
        {
            cout << "node " << j << " connects with " << " node: " << g[j]
[i].first << " weight: " << g[j][i].second << endl;
        }
    }

    return 0;
}
```

Dijkstra

```
#include <bits/stdc++.h>
using namespace std;
typedef vector<vector<pair<int, int>>> graph;
typedef pair<int, int> pii;
const int INF = 1e9;
vector<int> dijkstra(int start, graph &g)
{
    vector<int> dist(g.size(), INF);
    dist[start] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, start});
    while (!pq.empty())
    {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (d > dist[u])
            continue;
        for (auto edge : g[u])
        {
            int v = edge.first;
            int w = edge.second;
            if (dist[u] + w < dist[v])
            {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

DFS

```
#include <bits/stdc++.h>
using namespace std;

typedef vector<vector<int>> graph;

void dfs(int u, graph &g, vector<bool> &visited) {
    visited[u] = true;
    // process node u here
    for (int v : g[u]) {
        if (!visited[v]) {
            dfs(v, g, visited);
        }
    }
}

int main() {
    int n = 5; // number of nodes
    graph g(n);
    // Example edges
    g[0].push_back(1);
    g[0].push_back(2);
    g[1].push_back(3);
    g[1].push_back(4);

    vector<bool> visited(n, false);
    dfs(0, g, visited);

    return 0;
}
```

BFS

```
#include <bits/stdc++.h>
using namespace std;

typedef vector<vector<int>> graph;

void bfs(int start, graph &g, vector<bool> &visited) {
    queue<int> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        // process node u here
        for (int v : g[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}

int main() {
    int n = 5; // number of nodes
    graph g(n);
    // Example edges
    g[0].push_back(1);
    g[0].push_back(2);
    g[1].push_back(3);
    g[1].push_back(4);

    vector<bool> visited(n, false);
    bfs(0, g, visited);

    return 0;
}
```

```
#include <bits/stdc++.h>
using namespace std;

template <class T>
vector<int> kmp(T s){
    int n = s.size();
    vector<int> pi(n);
    pi[0] = 0;
    for(int i = 1; i < n; i++){
        int j = pi[i - 1];
        while(j > 0 && s[j] != s[i])
            j = pi[j - 1];
        if(s[j] == s[i]) j++;
        pi[i] = j;
    }
    return pi;
}

int main(){
    string a = "oso oso";
    for(int i : kmp(a)){
        cout << i<<" ";
    }

    return 0;
}
```

Exponenciación Binaria y Modulo

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int mod = 1e9 + 7;

int mul(int a, int b) {
    return ((a%mod)*(b%mod))%mod;
}

int add(int a, int b) {
```

```

        return ((a%mod) + (b%mod)) % mod;
    }

    int binary_exponentiation(int n, int e) {
        if(e == 0) return 1;
        int res = binary_exponentiation(n, e/2);
        if(e%2 == 0) {
            return mul(res, res);
        }
        return mul(n, mul(res, res));
    }

    signed main() {

        int n; cin >> n;

        int total = binary_exponentiation(2, n-2);

        int fact = 1;
        for(int i=n-2; i>=0; i--) {
            total = add(total, mul(binary_exponentiation(2, i), fact));
            fact = mul(fact, n - (i));
        }

        cout << total << endl;
        return 0;
    }

```

Binary Search

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> v = {4,5,8,9,10,32,64};

    int i = 0, j = v.size();
    int s;
    cin>>s;
    int pos = -1;
    while(i!=j){

        int m = (i + j)/2;
        if(s > v[m]){
            i = m + 1;
        }else if(s < v[m]){
            j = m - 1;
        }
        else{

```

```

        pos = m;
        break;
    }

}
cout << pos;
return 0;
}

```

VS Code Snippets

```

{
  "CP template": {
    "prefix": "CPT",
    "body": [
      "#include <bits/stdc++.h>",
      "using namespace std;",
      "",
      "int main() {",
      "    // Your code here",
      "    $1",
      "    return 0; ",
      "}"
    ],
    "description": "Competitive programming template"
  },
  "end line": {
    "prefix": "endl",
    "body": [
      "<< endl;"
    ],
    "description": "Output with newline"
  },
  "print vector": {
    "prefix": "pv",
    "body": [
      "for (auto &x : $1) {",
      "    cout << x << ' ' ;",
      "}",
      "cout << endl;"
    ],
    "description": "Print vector elements"
  }
}

```

Kruskal's Algorithm

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <tuple> // For adjacency list pair

using namespace std;

// --- 1. Edge Structure ---
struct Edge {
    int u, v; // Vertices of the edge
    int weight; // Weight of the edge

    // Comparator for sorting edges by weight
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

// --- 2. Disjoint Set Union (DSU) / Union-Find Structure ---
struct DSU {
    vector<int> parent;
    vector<int> rank;

    DSU(int n) {
        parent.resize(n + 1);
        iota(parent.begin(), parent.end(), 0);
        rank.assign(n + 1, 0);
    }

    int find_set(int i) {
        if (i == parent[i])
            return i;
        return parent[i] = find_set(parent[i]); // Path compression
    }

    void unite_sets(int a, int b) {
        a = find_set(a);
        b = find_set(b);

        if (a != b) {
            if (rank[a] < rank[b])
                swap(a, b);
            parent[b] = a;
            if (rank[a] == rank[b])
                rank[a]++;
        }
    }
};

```

```

// --- 3. Kruskal's Algorithm Function ---
long long kruskal(int n, vector<Edge>& edges) {
    // 1. Sort all edges by their weight
    sort(edges.begin(), edges.end());

    // 2. Initialize DSU structure
    DSU dsu(n);

    long long mst_weight = 0;
    int edges_count = 0;

    // 3. Iterate through all sorted edges
    for (const auto& edge : edges) {
        int u = edge.u;
        int v = edge.v;
        int weight = edge.weight;

        // Check if adding the edge connects two previously unconnected
        components
        if (dsu.find_set(u) != dsu.find_set(v)) {
            mst_weight += weight;
            dsu.unite_sets(u, v);
            edges_count++;

            if (edges_count == n - 1) {
                break;
            }
        }
    }
    return mst_weight;
}

// --- 4. Simple Usage Example (Main Function with Adjacency List) ---
int main() {
    int n = 4; // Number of vertices (1 to 4)

    // Adjacency List: key is vertex 'u', value is a list of pairs (v, weight)
    // Using 1-based indexing for vertices
    // Note: Since the graph is undirected, we add edges in both directions.
    vector<pair<int, int>> adj[n + 1];

    // Helper to add undirected edges
    auto add_edge = [&](int u, int v, int w) {
        adj[u].push_back({v, w});
        adj[v].push_back({u, w}); // For undirected graph
    };

    // Populate the Adjacency List (Same example graph as before)
    add_edge(1, 2, 10);
    add_edge(1, 3, 6);
    add_edge(1, 4, 5);
    add_edge(2, 4, 15);
    add_edge(3, 4, 4);
}

```

```

// --- Convert Adjacency List to Edge List ---
vector<Edge> edges;
for (int u = 1; u <= n; ++u) {
    for (const auto& neighbor : adj[u]) {
        int v = neighbor.first;
        int weight = neighbor.second;

        // Only add each edge once (e.g., if u < v) to avoid duplicates
        if (u < v) {
            edges.push_back({u, v, weight});
        }
    }
}

// --- Run Kruskal's Algorithm ---
long long mst_cost = kruskal(n, edges);

cout << "Graph built from Adjacency List:" << endl;
cout << "Number of vertices: " << n << endl;
cout << "Total weight of the Minimum Spanning Tree (MST): " << mst_cost <<
endl;
// Expected cost: 15

return 0;
}

```